The Knowledge Hub Universities

Parallel & Distributed Programming

Author:
Seif Rami

Student ID:
CU2200251

Contents

1 Coursework: Part 1 **1**
1.1 Question: A 1
1.2 Question: B 4
1.3 Question: C 4
1.4 Question: D 5
1.5 Question: E 5

2 Coursework: Part 2 **7**
2.1 Question: A 7
2.2 Question: B 8
2.3 Question: C 9

List of Figures

1 Part 1 Algorithm Question 1
2 Areas that generate work 2
3 Part 2 Algorithm Question 7
4 Part 2: DAG diagram 9

List of Tables

1 Performance Results for $n = 4096$ 6

1 Coursework: Part 1

Consider the following parallel program written using a fork–join model. The given algorithm computes the sum of an array using a recursive divide-and-conquer approach. The array is divided into two halves at each recursive step. One half is spawned as a parallel task, while the other half is executed by the current worker thread.

```
parallel_sum(A, n):
    if n == 1:
        return A[0]
    else:
        x = spawn parallel_sum(A[0 .. n/2 - 1], n/2)
        y = parallel_sum(A[n/2 .. n - 1], n/2)
        sync
        return x + y
```

Figure 1: Part 1 Algorithm Question

Assume that each addition operation takes 1 unit of time, the overhead of spawn and sync is non-negligible, and the input size is $n = 2^k$.

1.1 Question: A

Question: If the grain size is fixed to 1 (i.e., a new task is spawned for every single element), explain:

- Whether this is an efficient choice
- The effect on task creation overhead

Answer (Proof):

This is a divide-and-conquer binary tree. To determine whether choosing a grain size of $G = 1$ is efficient, we compare it with a larger grain size and analyse the difference in overhead.

Assume:

$$N = 4096 \quad \text{(Input size)}, \qquad G = 1 \quad \text{(Grain size)}$$

The number of leaf nodes is given by:

$$\text{Leaves} = \frac{N}{G}$$

The number of internal nodes in a full binary tree is:

$$\text{Internal Nodes} = \text{Leaves} - 1$$

Applying the formulas:

$$\text{Leaves} = \frac{4096}{1} = 4096$$

$$\text{Internal Nodes} = 4096 - 1 = 4095$$

Now we gather all the tasks that come together to calculate the total amount of useful work (work that gives us the result) and the total amount of overhead work (work that is used as a step to reach the result by managing parallel execution).

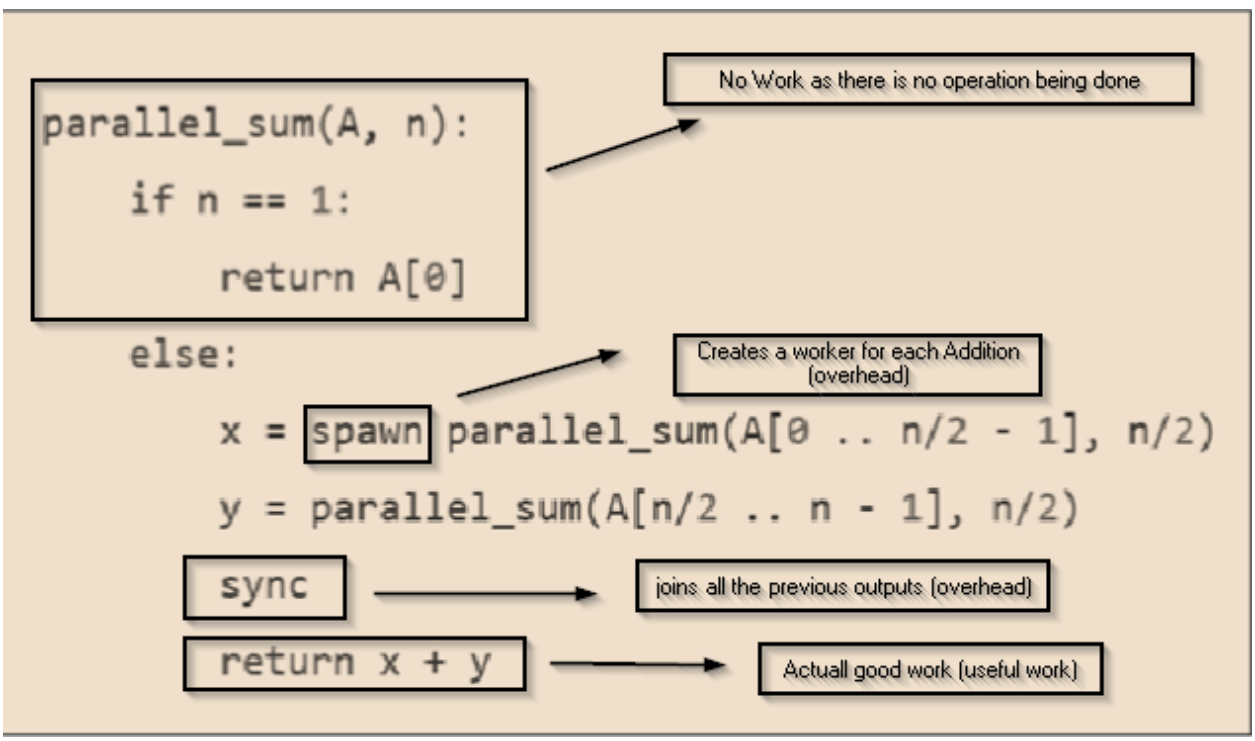

Figure 2: Areas that generate work

The total overhead operations are:

$$\text{Overhead} = \text{Spawns} + \text{Syncs} = 4095 + 4095 = 8190$$

The total useful computational work is:

$$\text{Useful Work} = \text{Additions} = 4095$$

If all operations are counted equally, the total number of operations performed is:

$$\text{Total Operations} = \text{Useful Work} + \text{Spawns} + \text{Syncs} = 4095 + 4095 + 4095 = 12285$$

So we conclude that with $N = 4096$ and $G = 1$, the program does an overall 12285 of total work (including overhead), 8190 of which is overhead, which is a lot of parallel management work, and moving on, we will compare with $G = 4$.

The total overhead operations are:

$$\text{Overhead} = \text{Spawns} + \text{Syncs} = 1023 + 1023 = 2046$$

The total useful computational work (counting only additions) is the sum of: (i) additions inside each leaf of size $G = 4$, and (ii) combine additions at internal nodes:

$$\text{Useful Work} = \underbrace{1024 \cdot (4-1)}_{\text{leaf additions}} + \underbrace{1023}_{\text{combine additions}} = 3072 + 1023 = 4095$$

If all operations are counted equally, the total number of operations performed is:

$$\text{Total Operations} = 4095 + 2046 = 6141$$

Therefore, for $N = 4096$ and $G = 4$, the algorithm performs 4095 useful additions and 2046 overhead operations. Compared to $G = 1$ (which incurs 8190 overhead operations), $G = 4$ reduces task-management overhead by a factor of 4, making it significantly more efficient. This answers the question that $G = 1$ is not an efficient choice and that it has more overhead going into computational management rather than computational output.

1.2 Question: B

Determine the span (T_∞) of the program as a function of n

Answer:

The formula for the span varies depending on the algorithm and its dependency structure. In this case, the algorithm is a divide-and-conquer algorithm, and the dependency structure follows a binary pattern where the problem is split into two equal parts at each recursive step. Since each split halves the input size, the span follows the level of the recursion tree, which determines the longest chain of dependencies.

$$T_\infty(n) = \log_2(n)$$

Assume:

$$N = 4096 \quad \text{(Input size)}, \qquad G = 1 \quad \text{(Grain size)}$$

Then:

$$T_\infty(4096) = \log_2(4096) = 12$$

1.3 Question: C

Determine the total work (T_1) of the program.

Answer: Total Work (Excluding Overhead)

Based on the previous calculations, the total work of the algorithm (excluding overhead) is computed as follows:

Assume:

$$N = 4096 \quad \text{(Input size)}, \qquad G = 1 \quad \text{(Grain size)}$$

The total work is defined as:

$$T_1(n) = n - 1$$

Substituting $n = 4096$:

$$T_1(4096) = 4096 - 1$$

$$T_1(4096) = 4095$$

Therefore, the total work of the algorithm is:

$$T_1(4096) = 4095$$

1.4 Question: D

Compute the parallelism of the algorithm

Answer:

Parallelism is defined as the ratio between total work and span and provides us with the maximum possible speed-up if we had unlimited processors:

$$\text{Parallelism} = \frac{T_1(n)}{T_\infty(n)}$$

From previous calculations:

$$T_1(4096) = 4095$$

$$T_\infty(4096) = 12$$

Therefore:

$$\text{Parallelism} = \frac{4095}{12}$$

$$\text{Parallelism} = 341.25$$

1.5 Question: E

Implement in C/C++ this algorithm with the best value of grain size (which can be determined from either theory or practice), and then use Cilkview / OpenMP to collect the following information of the whole program with n = 4096 or larger: Work (instructions) Span (instructions) Burdened span (instructions) Parallelism Burdened parallelism as well as the speed-up estimated on 2, 4, 8, 16, 32, 64 and 128 processors, respectively.

Answer: This program is a divide and conquer algorithm similar to the picture above, where the input N = 4096 and the best grain size is calculated through this formula.

$$g = \frac{N}{2P}$$

Assuming 8 cores are used:

$$g = \frac{4095}{2 \times 8} = 256$$

This is the best span, Assuming amount of processors is 8 and N = 4095

Results of the code:

Table 1: Performance Results for $n = 4096$

Threads	Time (s)	Speedup	Sum	Burdened Span	Burdened Parallelism
1	2.06×10^{-5}	1.000	4096	4	1023
2	3.60×10^{-4}	0.057	4096	20	204
4	1.64×10^{-4}	0.126	4096	36	113
8	3.23×10^{-4}	0.064	4096	68	60
16	5.80×10^{-4}	0.036	4096	132	31
32	1.29×10^{-3}	0.016	4096	260	15
64	2.43×10^{-3}	0.009	4096	516	7
128	4.64×10^{-3}	0.004	4096	1028	3

2 Coursework: Part 2

Consider the following Cilkview++ code fragment:

```
static const int COUNT = 4;
static const int ITERATION = 1000000;
long arr[COUNT];
long do_work(long k){
  long x = 15;
  static const int nn = 87;
  for (long i = 1; i < nn; ++i)
    x = x / i + k % i;
  return x;
}
int cilk_main(){
  for (int j = 0; j < ITERATION; j++)
    cilk_for (int i = 0; i < COUNT; i++)
      arr[i] += do_work( i + j);
}
```

Figure 3: Part 2 Algorithm Question

2.1 Question: A

Estimate the (theoretical) parallelism of this program

Answer:

To acquire the parallelism of this algorithm, we must use this formula:

$$\text{Parallelism} = \frac{T_1(n)}{T_\infty(n)}$$

Based on the formula, we need to acquire the work. The algorithm starts with a *do_work* function that takes an input and does some math on it, which returns the same input as is to simulate empty work. In the main function, we have two nested loops, the first iterates over 1000000 times and the other ones 4 times and inside is the insertion into the array arr

using the *do_work* function with the input of the indexes of the array, which gives the work to be the first loop index times the second:

$$T_1 = Loop1 \times Loop2$$

$$T_1 = 1000000 \times 4 = 4000000$$

Now we must acquire the span, and since there are two for loops, the first one is sequential, meaning we have 1,000,000 units of work. The second one is parallel, and based on the unit of time that we get from the loop, we then multiply it by the sequential work, meaning :

$$T_\infty = 1000000 \times Nunitsoftime$$

And if we assume it's 1 unit of time, then;

$$T_\infty = 1000000 \times 1 = 1000000$$

Finally, we use the parallelism formula:

$$\text{Parallelism} = \frac{4000000}{1000000} = 4$$

$$\text{Parallelism} = 4$$

2.2 Question: B

Explain why the burdened span of this program is much higher than the (theoretical) span.

Answer:

Well The span is always the actual work that is needed to get the intended results, while the burdened span is the real-life scenario as it has overhead work, which includes the work to create workers to parallelise the program, as well as the work to sync all the outputs. These are the steps that are needed to manage parallelism, which span ignores.

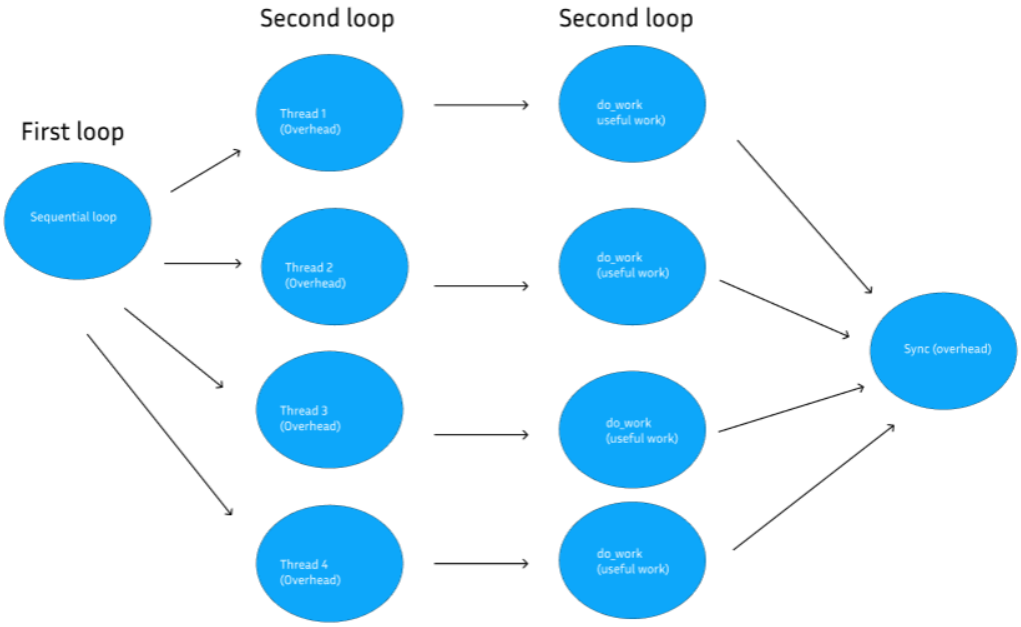

Figure 4: Part 2: DAG diagram

Finally, since the program iterates over 1000000 times, with each iteration, workers are spawned to sum the indexes of both loops, making the overhead enormous, as mentioned in the DAG diagram, each time we get four worker threads for over 1000000.

2.3 Question: C

How can we fix the performance bottleneck of this program?

Answer:

We can parallelise the first loop as it has the highest number of iterations and make the second also parallelised or sequential based on which is more efficient. Finally, another alternative is just make the initial loop count = 4 iterations and the nested one be 1000000 iterations.